



Security Audit

Ithaca 4th (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	21
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Ithaca team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

The Ithaca Protocol is a decentralized finance (DeFi) platform that aims to enhance liquidity provision and decentralized governance within the DeFi ecosystem. It employs advanced automated market making (AMM) mechanisms to facilitate seamless trading and liquidity management while incorporating a governance model that empowers stakeholders in decision-making processes. The protocol focuses on efficient liquidity pool management, attractive incentives for liquidity providers, and robust security measures to ensure the protection of user funds. Continuously evolving through its development roadmap, Ithaca Protocol integrates with other DeFi projects to expand its ecosystem and drive sustainable growth.

Project Name: Ithaca

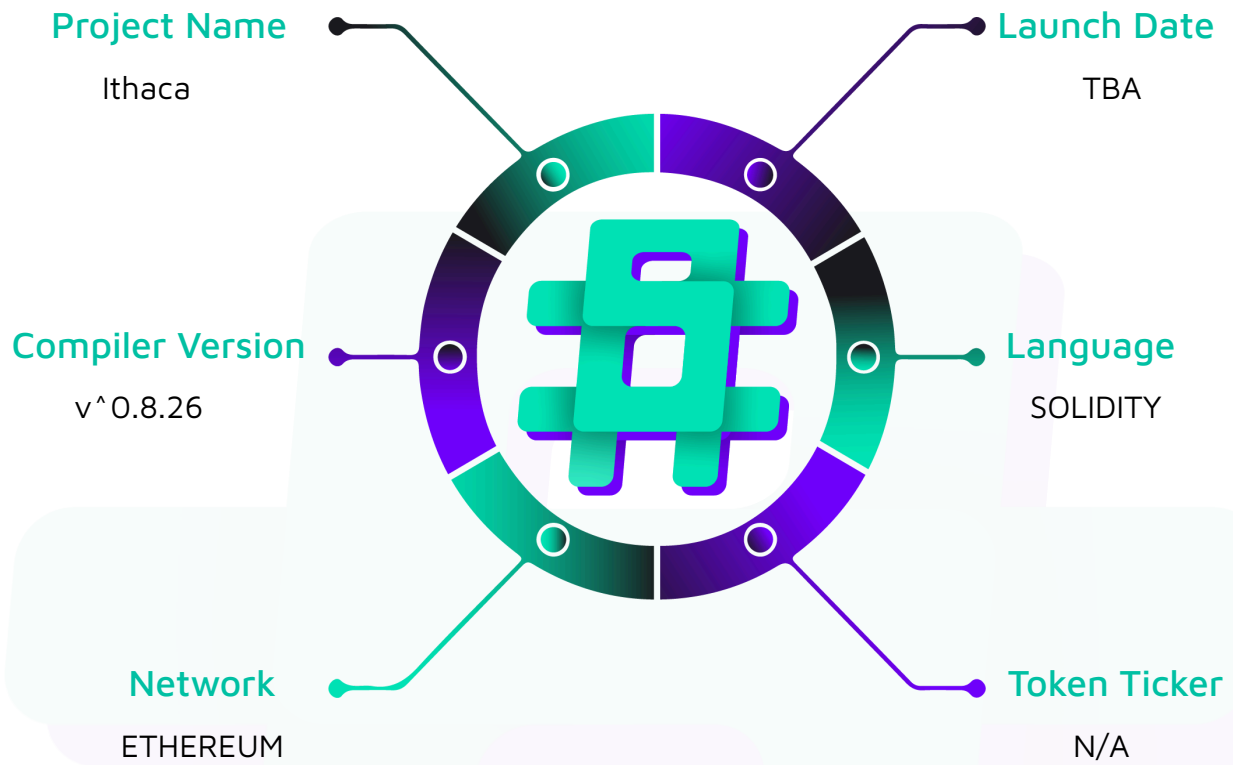
Project Type: Defi

Compiler Version: ^0.8.26

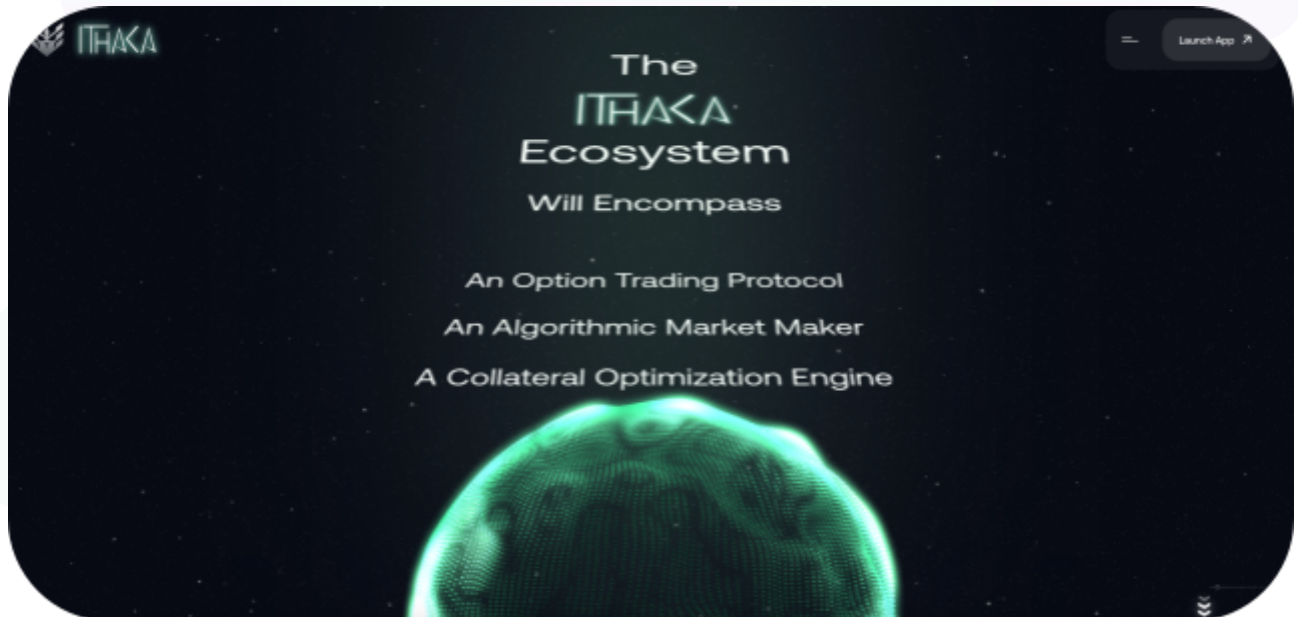
Website: <https://www.ithacaprotocol.io/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Ithaca project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Ithaca Smart Contracts
Platform	Ethereum / Solidity
Audit Date	May, 2025
Contract 1	AaveV2DepositManager.sol
Contract 2	DepositManager.sol
Contract 3	Factory.sol
Audited GitHub Commit Hash	1011b721c3c86e1736ea2fae7b4062d6357f87a5
Fix Review GitHub Commit Hash	10dc9a10baa826c684736f11b7f31fe7dfb9720b

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerabilities

2 Low severity vulnerabilities

1 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
DepositManager.sol Allows users to: - Deposit assets to receive fund tokens representing their share - Request withdrawals by burning fund tokens (processed in queue order) - Update external asset balances with oracle signatures - View their active withdrawal requests and status - Preview deposit amounts and conversion rates - Check withdrawal queue status and pending releases Allows managers to: - Transfer management rights to another address - Schedule fee changes (with 10-day timelock) - Collect AUM and performance fees automatically Allows owners (Factory) to: - Set rewards fee percentage for protocol - Update oracle threshold for staleness checks - Pause/unpause the contract for emergencies - Change Ithaca delegate address - Set high watermark reset period - Initialize new contract versions	Contract achieves this functionality.

AaveV2DepositManager.sol Allows users to: - Deposit assets directly into Aave V2 to receive fund tokens - Withdraw assets immediately by burning fund tokens (no queue) - Convert between assets and shares at current exchange rate - Check total assets deposited in Aave (aToken balance) - View current exchange rate between fund tokens and assets Allows managers to: - Transfer management rights to another address Allows owners to: - Pause/unpause the contract for emergencies	Contract achieves this functionality.
Factory.sol Allows users to: - Deploy new trading agents with custom parameters (name, symbol, fees) - Make initial deposit during agent deployment - Query all agents, agents by manager, or agents by token holder - View agent count and paginated agent lists Allows owners to: - Upgrade all DepositManager instances via beacon proxy	Contract achieves this functionality.

- | | |
|--|--|
| <ul style="list-style-type: none">- Batch initialize instances after upgrades- Set minimum deployment amount requirement- Update oracle and delegate addresses globally- Pause/unpause the factory and individual DepositManagers- Configure DepositManager parameters (rewards fee, update threshold)- Schedule and execute fee changes for specific agents- Set watermark reset periods for agents- Manage delegate settings for individual DepositManagers | |
|--|--|

Code Quality

This audit scope involves the smart contracts of the Ithaca project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Ithaca project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] DepositManager/AaveV2DepositManager/Factory#_deposit/_processWithdrawals/deployAgent - Contract does not account for fee-on-transfer tokens.

Description

DepositManager.sol and AaveV2DepositManager.sol and Factory.sol, all of the contract transfer asset tokens, but do not consider the possibility of fee-on-transfer token transfer.

Vulnerability Details

The inherent assumption while transferring asset tokens is that it won't charge upon transferring tokens.

However, under a case where asset token is fee-on-transfer, the amount transferred will be less than what the contract will account for.

Here some cases where tokens transfer might cause revert or error in accounting.

```
function deployAgent(string memory name, string memory symbol, uint256 amount, uint256
aumFee, uint256 performanceFee) external whenNotPaused nonReentrant returns (address
token, address depositManager) {
    if (defaultAssetToken == address(0)) revert DefaultAssetTokenNotSet();
    if (minDeployAmount > 0 && amount < minDeployAmount) revert
InsufficientDeployAmount();
    if (amount > 0) {
        IERC20(defaultAssetToken).transferFrom(msg.sender, address(this), amount);
    }
    token = address(new Token(name, symbol, lzEndpoint, address(this)));
    bytes memory initData =
abi.encodeWithSelector(DepositManager.initialize.selector, defaultAssetToken, token,
msg.sender, address(ithacaVault), delegate, aumFee, performanceFee);
```



```

        depositManager = address(new BeaconProxy(address(depositManagerBeacon),
initData));
        Token(token).setMinter(depositManager);
        getDepositManager[token] = depositManager;
        isAgent[token] = true;
        agents.push(token);
        if (amount > 0) {
            IERC20(defaultAssetToken).approve(depositManager, amount);
            DepositManager(depositManager).deposit(msg.sender, amount);
        }
        emit AgentDeployed(token, depositManager, defaultAssetToken, name, symbol,
msg.sender);
    }

```

Deployment will not happen since `deposit` into the manager will be done through `amount` instead of `actual balance`. And if there is more balance in the contract, then that will be accounted for by `msg.sender` instead.

Similarly we have `_deposit` and `_processWithdrawals` that transfer assets.

Impact

Error in accounting and possible reverts.

Recommendation

Check the balance of the contract before and after the transfer. Subtract the amounts and that will be the final balance to consider.

Status

Resolved

Low

[L-01] DepositManager/AaveV2DepositManager/Factory# - Lack of gap variable in contracts will lead to storage collision

Description

The contracts are intended to be upgradable but do not have storage gaps. This could lead to storage collisions in future upgrades.

Vulnerability Details

Even though all the contracts are designed to be upgradable, they lack storage gaps. `Factory.sol` and `DepositManager.sol` contain 2 unused slot variables. But those are not enough in case there is a need to add more variables post upgrade.

Impact

Risk of storage collision after upgrade.

Recommendation

Include a standard `_gap` variable instead.

Status

Acknowledged

[L-02] AaveV2DepositManager#deposit - An attacker can permanently brick AaveV2DepositManager when fund token and asset token have same decimals

Description

Consider a scenario where fund token and asset tokens have the same decimals. In that case, an attacker can perform a standard inflation attack and subsequent deposits will be bricked.

Vulnerability Details

Under the condition when `fundsTokens` and `assetTokens` have the same decimal places, it is possible to pull off inflation attack over the contract by Depositing 1 wei worth of asset to get 1 share back , directly transfer 100 `aTokens` the contract. `aTokens` are tradable , and an attacker can get them by directly supplying `usdc` to `aave`.

```
function _deposit(address receiver, uint256 assets) internal returns (uint256 shares)
{
    if (assets == 0) revert InvalidAmount();

    if (receiver == address(0)) revert ZeroAddress();

    shares = convertToShares(assets);

    if (shares == 0) revert InvalidAmount();

    asset.safeTransferFrom(msg.sender, address(this), assets);

    aaveLendingPool.deposit(address(asset), assets, address(this), 0);

    fundToken.mint(receiver, shares);

    emit Deposit(receiver, assets, shares);
}
```

Due to this condition :

```
if (shares == 0) revert InvalidAmount();
```

Any subsequent deposit will revert since share calculation will round down to zero.

Proof of Concept

Paste the following POC in in `AaveV2DepositManager.t.sol` Also create user2 and mint user1 some aUSDC first.

```
function testInflationAttack() public {

    uint256 amount = 100e6;

    vm.startPrank(user);

    uint256 shares = aaveDepositManager.deposit(user, 1);

    console.log("user1 share", shares);

    aUSDC.transfer(address(aaveDepositManager), 100e6);

    vm.stopPrank();

    // confirm share calculation rounds to zero

    uint256 inflatedShares = aaveDepositManager.convertToShares(100e6);

    console.log("share after inflation" , inflatedShares);

    vm.expectRevert("InvalidAmount");

    vm.prank(user2);

    uint256 shares2 = aaveDepositManager.deposit(user2, 100e6);

    console.log("user2 shares", shares2); }
```

Impact

The entire contract deposit process is bricked.

Recommendation

Consider implementing dead shares in share calculation so that rounding to zero is not possible.

Status

Resolved

QA

[Q-01] Sio2AdapterAssetManager - Remove unnecessary if condition.

Description

The `calculateAccruedFees` function has a stale `else` condition that will not execute.

```
if (lastFeesCollectionTimestamp > 0) {  
    timeSinceLastCollection = block.timestamp - lastFeesCollectionTimestamp;  
} else { //@audit - This condition will never be executed since after first  
deposit, lastFeeCollectionTimestamp will be > 0  
    timeSinceLastCollection = 0;  
    return (0, block.timestamp);  
}
```

The `else` condition will never be executed after the first deposit is done.

Recommendation

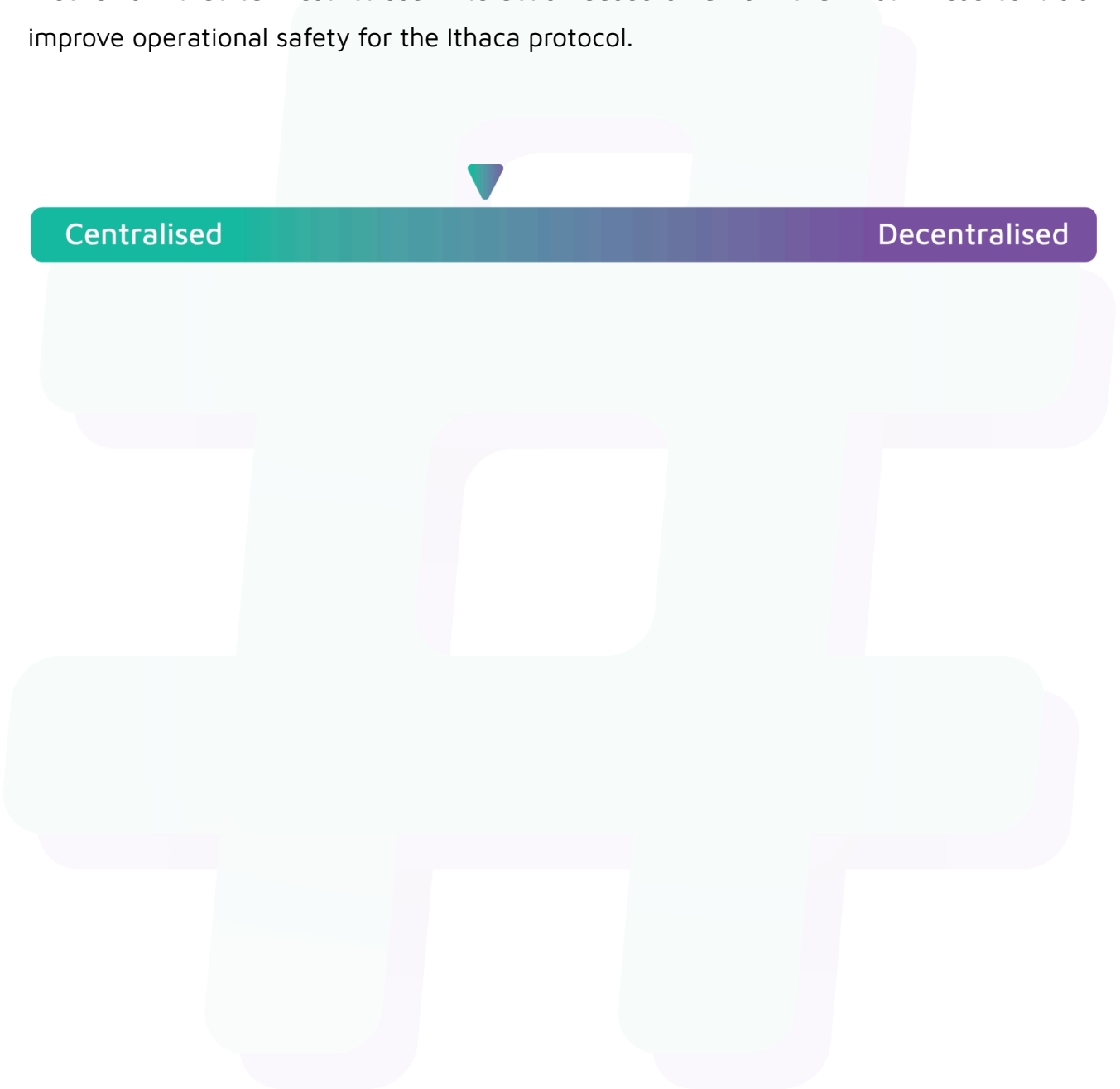
Remove the unnecessary `else` condition.

Status

Resolved

Centralisation

The Ithaca protocol includes several centralizing elements that enhance control and security but reduce decentralization. Specifically, there are owner and manager roles with privileged access to critical functions, as well as an externally managed sanctions mechanism that can restrict user interaction based on an off-chain list. These controls improve operational safety for the Ithaca protocol.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Ithaca project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.